

On Computing Signal Probability and Detection Probability of Stuck-at Faults

Sreejit Chakravarty and Harry B. Hunt III

Abstract—Algorithms for the following two problems are presented: 1) computing detection probability of stuck-at faults (CDP) and 2) computing signal probability (CSP). These problems arise in the context of random testing, pseudorandom testing, and testability analysis of combinational circuits.

The algorithm for CDP presented here combines the notion of supergates [21], [22] and a refinement of the algorithm for CDP presented in [10]. This algorithm for CDP can be used to compute the exact value of detection probability of multiple stuck-at faults in circuits with multiple outputs. Single-input, single-output pseudo gates are inserted to model stuck-at faults and derive an "equivalent" single-output circuit. CDP is thus reduced to the problem of computing the probability distribution of the output over the set of four logic values $\{0, 1, d, \bar{d}\}$.

The algorithm for CDP uses an efficient "enumeration algorithm." We show how this "enumeration algorithm" can be used to refine the algorithm for CSP presented in [21], [22].

Index Terms—Built-in self-test, detection probability, pseudorandom testing, signal probability, testability analysis.

I. INTRODUCTION

WE present algorithms for the following two problems: 1) CDP and 2) CSP. CSP and CDP are defined below.

CDP: Given a combinational circuit C and a stuck-at fault F , compute the probability that a randomly chosen input vector detects the fault F , under the assumption that every input vector is chosen with equal probability.

CSP: Given a combinational circuit C , compute the probability that a randomly chosen input vector sets the output of C to 1, under the assumption that every input vector is chosen with equal probability.

CDP arises in the context of random testing as well as pseudorandom testing as discussed in [11], [20], and [23]. CSP arises in the context of testability analysis. Testability analysis involves the computation of "Testability Measures." One-controllability and zero-controllability, defined below, are two testability measures that have been proposed in [21], [22].

One-Controllability (Zero-Controllability): of a line l , denoted by $C1(l)$ ($C0(l)$), is the probability of line l having a value 1 (0) on a randomly selected vector.

The use of these testability measures is discussed in [2] and [3]. From the above definitions it follows that the problem of

computing zero-controllability and one-controllability are equivalent to CSP. In the discussion to follow, we need the following definitions.

One-Observability (Zero-Observability): of a line l , denoted by $B1(l)$ ($B0(l)$), is the probability of observing the value of line l at a primary output when the value of line l is 1 (0).

Note that if $D0(l)$ and $D1(l)$ are the detection probability of line l stuck-at 0 and 1, respectively, then we have the following relation.

$$D0(l) = C0(l) \times B0(l) \quad (1)$$

$$D1(l) = C1(l) \times B1(l). \quad (2)$$

It has been known for sometime that CDP and CSP are #P-Hard [8]–[10], [16]. This implies that even if $P = NP$ there may still not exist a polynomial time algorithm for CDP. We therefore seek an algorithm that will not be exponential all the time. In Sections II–IV, we discuss such an algorithm for CDP. The algorithm for CDP presented here uses the notion of supergates [21], [22] and a refinement of the algorithm for CDP presented in [10] to process the supergates. Before we review the existing work on CDP we note the following features of our algorithm: 1) it computes the exact value of detection probability; 2) it can be used not only for single stuck-at faults but also for multiple stuck-at faults; and 3) it can be used for computing the exact value of the detection probability of stuck-at faults in circuits with multiple outputs. In [26] and [27], we show how this algorithm can be extended to compute detection probability of nonfeedback bridging faults and for a number of other problems.

Algorithms for CDP have been discussed in [15], [20]–[22], [25]. All these algorithms are for single stuck-at faults only. The Cutting Algorithm [20] actually computes only signal probabilities. In [20] it was shown how estimates of detection probability can be made using signal probability. Experimental results in [13] show that the Cutting Algorithm very often computes the detection probability of the fault to be 0 even if the actual value of the detection probability is much larger. Thus, the Cutting Algorithm does not compute the exact value of the detection probability of faults.

The problem addressed by STAFAN [15] is defined as follows: given a set S of input vectors, for each single stuck-at fault F determine the number P of input vectors of S that detect the fault F . By uniformly sampling the set of input vectors we get a sampling technique for estimating the detection probability of faults. Note that $\frac{P}{N}$, where P is as computed above and N is the total number of input vectors, is the detection probability of

Manuscript received February 24, 1988; revised October 15, 1988. A preliminary version of this work was reported in [10]. The research of both authors was supported by NSF Grant DCR84-03014.

S. Chakravarty is with the Department of Computer Science, the State University of New York, Buffalo, NY 14260.

H. B. Hunt III is with the Department of Computer Science, the State University of New York, Albany, NY 12222.

IEEE Log Number 9037989.

F if and only if the set S is the set of all the input vectors. Therefore, extensions of STAFAN for CDP will be exponential in the number of inputs to the circuit. This is true for all circuits and such an algorithm is clearly not practical. We show that our algorithm will never perform worse than such an algorithm that processes all the input vectors and often does substantially better.

PREDICT [22] also addresses the problem of CDP. It uses expressions (1) and (2) for $D0(l)$ and $D1(l)$ stated above. Expressions (1) and (2) can be used for computing the exact value of the detection probability of faults if and only if we can come up with algorithms for computing the exact values of the observabilities and controllabilities defined above. In [22] an algorithm for computing the exact value of controllabilities is discussed. However, the algorithm for computing observabilities presented in [22] does not compute the exact value of the observabilities. Therefore, the algorithm in [22] cannot be used for computing the exact value of the detection probability of faults. In [21] an attempt was made to modify the algorithm used in PREDICT so that the exact value of the detection probability of single stuck-at faults can be computed. However, the algorithm in [21] can compute the exact value of the detection probability of faults only for some very restricted cases. We substantiate this point in Section IV.

PROTEST [25] is a tool for estimating detection probability of single stuck-at faults. It uses "single path sensitization" and is therefore unable to compute the exact value of detection probability.

In Section V we show that our algorithm for CDP, with a minor modification, can also be used for computing the exact value of signal probability. Algorithms for CSP are discussed in [16]–[18], [20], [21], and [25]. The algorithms in [16], [20], and [25] compute only approximate values of signal probability. The exponent suppression method [17], [18] is not a useful algorithm because it very often requires exponential space. Our algorithm for CSP has similarities with the algorithm in [21]. However, we show that an efficient enumeration algorithm, identical to the one used for CDP, can be used to refine and improve the efficiency of the algorithm for CSP presented in [21].

II. A FORMULATION OF CDP

CDP is formulated using the notion of *pseudo gates* and four-valued logic. The four logic values are $\{0, 1, d, \bar{d}\}$ [19]. The interpretation of these four logic values is shown in Table I. Consider a line l in the circuit C that connects points F and G as shown in Fig. 1. Assume that l is stuck-at 0. The behavior of l in the faulty and fault-free circuit, for the same input vector, is described in Table II. This behavior of a line stuck-at 0 is abstracted by a pseudo gate α_0 . The d -cubes [19] of α_0 are shown in Table III, where F is the gate input and G is the gate output. The behavior of a line stuck-at 1 can be similarly abstracted by a pseudo gate α_1 whose d -cubes are shown in Table III.

We need the following circuit modification step, henceforth referred to as ModI.

ModI: If a line between points Z_1 and Z_2 is stuck-at 0 (stuck-at 1) then modify C by inserting a pseudo gate α_0 (α_1) with input Z_1 and output Z_2 .

Consider the circuit obtained after applying ModI to the given circuit. If the circuit consists of only one output Y then the detection probability of the fault equals the probability of $\{(Y = d) \vee (Y = \bar{d})\}$ computed under assumptions A1, A2, and A3

TABLE I

Faulty Value	Fault-Free Value	Four-Valued Logic
0	0	0
0	1	d
1	0	$\bar{d}$
1	1	1

stated below. If the circuit consists of more than one output $Y_1, \dots, Y_K$ then the detection probability of the fault equals the probability of the event E , defined below,

$$E = \bigvee_{i=1}^K \{(Y_i = d) \vee (Y_i = \bar{d})\} \quad (3)$$

where the probability of E is computed under assumptions A1, A2, and A3.

A1) Probability of the events $\{X_i = 0\}$ and $\{X_i = 1\}$ equals $\frac{1}{2}$, for all input variables X_i .

A2) Probability of the events $\{X_i = d\}$ and $\{X_i = \bar{d}\}$ equals 0 for all input variables X_i .

A3) Let $X_1, \dots, X_n$ be the set of input variables and $a_1, \dots, a_n$ be a set of constants from $\{0, 1, d, \bar{d}\}$. Then,

$$\text{Prob}\{(X_1 = a_1) \wedge \dots \wedge (X_n = a_n)\} = \prod_{i=1}^n \text{Prob}\{X_i = a_i\}.$$

For circuits with two or more primary outputs, we use another circuit modification step discussed below. Attach the *pseudo circuit* shown in Fig. 2 to the outputs $Y_1, \dots, Y_K$. Table IV defines the pseudo gate Θ . In Table IV, Θ_1 and Θ_2 are the two inputs, Θ_3 is the output of the gate, and X denotes a don't care condition. An input is said to satisfy the don't care condition if it has any one of the four values from the set $\{0, 1, d, \bar{d}\}$.

Observe that the output Y of the pseudo circuit is set to d if and only if at least one of the inputs $Y_1, \dots, Y_K$ has the value d or $\bar{d}$. Thus, the event E defined above is equivalent to the event that Y , the output of the pseudo circuit, equals d . The *probability distribution* of a point Z is the set of probability of the following events: $\{Z = 0\}$, $\{Z = 1\}$, $\{Z = d\}$, $\{Z = \bar{d}\}$. The probability expressions of a gate are necessary for computing the probability distribution of the gate output from the probability distribution of the gate inputs. The probability expressions are derived from the d -cubes of the respective gates. Our algorithm requires the probability expressions for each of the following gates α_0 , α_1 , Θ , AND, OR, and NOT gates. These probability expressions are listed in the Appendix.

An example will help clarify the formulation presented here. The augmented circuit for the fault of Fig. 3(a) is shown in Fig. 3(b). The values at Z_3 for each of the input vectors of the circuit are shown in Table V. Such tables can be computed from the d -cubes of the gates comprising the circuit. From Table V we see that the value of Z_3 is $d, \bar{d}$ for only one out of a total of eight input vectors. Therefore, the detection probability of the fault of Fig. 3(a) is $\frac{1}{8}$.

III. BASIC STEPS OF THE ALGORITHM

Here we discuss the basic steps of our algorithm for CDP. Before we can enunciate the basic steps we need some definitions.

For each line X in the circuit define a *supergate* [21] $SG(X)$ as follows. The *supergate* of a line X in the circuit is the smallest predecessor subnetwork feeding X whose inputs are logically independent, i.e., have no signal correlation. A more formal definition of supergates is discussed in [21]. A supergate

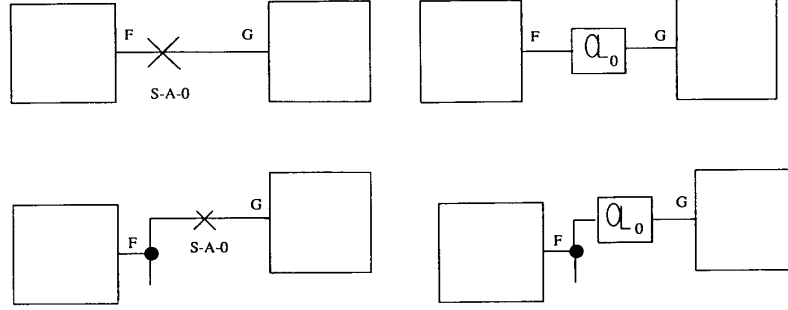


Fig. 1. Insertion of pseudo gates.

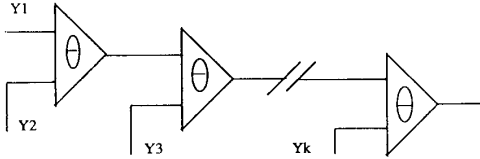


Fig. 2. Reduction of multiple outputs to a single output.

TABLE II

Fault-Free Circuit		Faulty Circuit	
F	G	F	G
0	0	0	0
0	0	1	0
1	1	0	0
1	1	1	0

TABLE III

gate	F	G
α_0	0	0
	$\bar{d}$	0
	$\bar{d}$	$\bar{d}$
	1	$\bar{d}$
α_1	0	$\bar{d}$
	$\bar{d}$	$\bar{d}$
	$\bar{d}$	1
	1	1

TABLE IV

Θ_1	Θ_2	Θ_3
$\bar{d}$	X	$\bar{d}$
X	$\bar{d}$	$\bar{d}$
$\bar{d}$	X	$\bar{d}$
X	$\bar{d}$	$\bar{d}$
0	0	1
0	1	1
1	0	1
1	1	1

is said to be a *maximal supergate* if it is not properly contained in a larger supergate. The maximal supergates of the circuit of Fig. 4 are demarcated by dashes. Let m be the number of gates in the circuit. Then, as discussed in [21], there exists an $O(m^2)$ algorithm for computing the maximal supergates of a circuit.

The *reduced circuit graph* (RCG) of a circuit is a graph $G(V_1, E_1)$ where V_1 is the set of maximal supergates of the circuit. Also, there exists an edge from node $SG(X)$ to node $SG(Y)$ if and only if X is an input of $SG(Y)$. After the maximal supergates of a circuit have been identified, the RCG of the circuit can be constructed in linear time. Fig. 5 depicts the RCG of the circuit in Fig. 4. The following lemma is from [21].

Lemma 3.1: The RCG of any circuit is a tree, i.e., for every pair of nodes in RCG there is no more than one directed path from one node to the other node.

Our algorithm requires a *total ordering of the maximal supergates* of the circuits. Since the RCG is a directed acyclic graph, a total ordering of the supergates can be derived by topologically sorting the RCG [4]. Define *SG-PROB* as follows.

SG-PROB: Given a supergate $SG(X)$ and the probability distribution of the inputs of $SG(X)$, compute the probability distribution of the output of $SG(X)$, i.e., the point X .

The *basic steps* of our algorithm for computing detection probability are stated below.

- 1) Augment the circuit as discussed in Section II.
- 2) Let $Z_1, \dots, Z_K$ be the points in the augmented circuit such that for all $1 \leq i \leq K$, $SG(Z_i)$ is maximal. Compute the maximal supergates $SG(Z_1), \dots, SG(Z_K), SG(Y)$ of the augmented circuit.
- 3) Compute a total order of the maximal supergates of the circuit. Without loss of generality, assume that the total order of the supergates is $SG(Z_1), \dots, SG(Z_K), SG(Y)$. Note that $SG(Y)$ will always be the maximal element of any total ordering of the supergates. FOR $I = 1$ to K Solve SG-PROB for the supergate $SG(Z_I)$.
- 4) Solve SG-PROB for the supergate $SG(Y)$. Detection probability equals the sum of the probability of $\{Y = d\}$ and $\{Y = \bar{d}\}$.

The reader should note the following points.

- 1) At the beginning of the I th iteration in step 3, the probability distribution of lines $Z_1, \dots, Z_{I-1}$ and the primary inputs to the circuit are known. From the definition of the total order of the supergates it follows that each input of the supergate $SG(Z_I)$ is either: 1) one of the lines $Z_1, \dots, Z_{I-1}$; or 2) a primary input of the circuit. Therefore, during the I th iteration of step 3, the probability distribution of all the inputs of $SG(Z_I)$ are known.

- 2) Augmented circuits, defined above, have only one output Y . In the total order, $SG(Y)$ is always the maximal element. At the end of step 3, the probability distribution of all the inputs of the supergate $SG(Y)$ has been computed. This explains step 4.

The example of Fig. 4 is used to illustrate the basic steps of the algorithm. The total ordering of the supergates used is $SG(Z_1), SG(Z_2), SG(Z_3), SG(Y)$. The probability distributions of Z_1, Z_2, Z_3 , and Y , which are computed in that order, are tabulated in Table VI. From the probability distribution of Y , as shown in Table V, it follows that the detection probability of the fault in our example is $\frac{1}{128}$.

IV. AN ALGORITHM FOR SG-PROB

In Section III we had seen how an algorithm for SG-PROB can be used for computing detection probability. In this section, we present an algorithm for SG-PROB and henceforth refer to it as H-ASS.

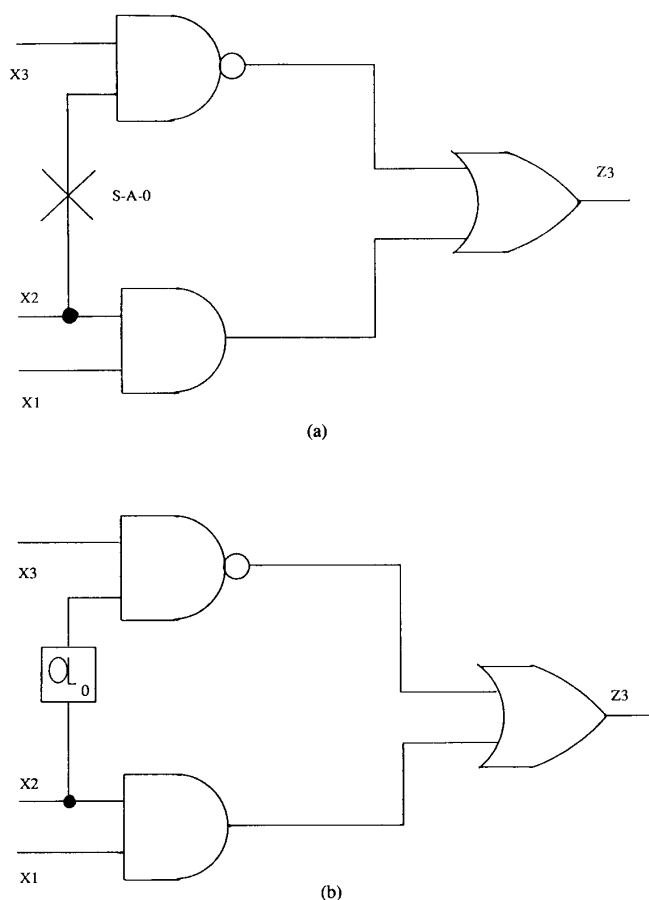


Fig. 3. An example of using pseudo gates.

TABLE V

X_3	X_2	X_1	Z_3
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	$\bar{d}$
1	1	1	1

H-ASS uses the notion of *tree decomposition* of a supergate. A circuit T is said to be a *tree circuit* if and only if all fan-out points of T are primary inputs of T . A tree subcircuit T of a supergate C is said to be a *maximal tree subcircuit (MTS)* if and only if there does not exist any other tree subcircuit T' of C such that T is a subcircuit of T' . The MTS's of the supergate of Fig. 6 are demarcated using dashes.

The tree decomposition of a circuit is unique for the following reason. Let FP be the set consisting of the fan-out points of the supergate and the output of the supergate. Then, there is a one-to-one correspondence between the MTS's of the supergate and the members of FP. For every $R \in \text{FP}$, let R^T denote the corresponding MTS. Then, R is the output of R^T .

Our algorithm requires a *total ordering of the fan-out points* of the supergate. The total ordering of the fan-out points is computed by topologically sorting the *fan-out graph (FG)* of

the supergate. The FG G of a supergate is defined as follows: $G = (V_1, E_1)$; V_1 is the set of fan-out points of the supergate; there exists an edge between nodes X and Z of V_1 if and only if X is an input of Z^T , the MTS with output Z . The FG of the circuit of Fig. 6 is shown in Fig. 7 from which we see that $\langle R_1, R_2, R_3, R_4 \rangle$ is a total ordering of the fan-out points.

Let $\langle R_1, \dots, R_K \rangle$ be a total ordering of the fan-out points of the supergate. Then, $\langle R_1 = a_1, \dots, R_K = a_K \rangle$ is an *H-Assignment* if and only if for all i , $a_i \in \{0, 1, d, \bar{d}\}$. An H-Assignment is said to be *consistent* if and only if the probability of the event $\{R_1 = a_1, \dots, R_K = a_K\}$ is greater than zero. For example in Fig. 6 let the total order of the fan-out points be $\{R_1, R_2, R_3, R_4\}$. The H-Assignment $\langle R_1 = d, R_2 = 0, R_3 = 0, R_4 = 1 \rangle$ is an inconsistent H-Assignment whereas $\langle R_1 = 0, R_2 = 1, R_3 = 0, R_4 = 1 \rangle$ is a consistent H-Assignment.

Our algorithm requires the computation of some conditional probability distributions which is discussed in Lemma 4.1.

Lemma 4.1: Let X^T be a tree circuit, X be the output of X^T , $\{X_1, \dots, X_K\}$ be the inputs of X^T that fan out, $\{Y_1, \dots, Y_P\}$ be the inputs of X^T that do not fan out, $\{a_1, \dots, a_K\}$ be a set of constants from $\{0, 1, d, \bar{d}\}$, and m be the number of gates in X^T . Then, for all $a \in \{0, 1, d, \bar{d}\}$ the probability of the following event can be computed using $O(m)$ arithmetic operations.

$$\{X = a \parallel X_1 = a_1, \dots, X_K = a_K\}. \quad (4)$$

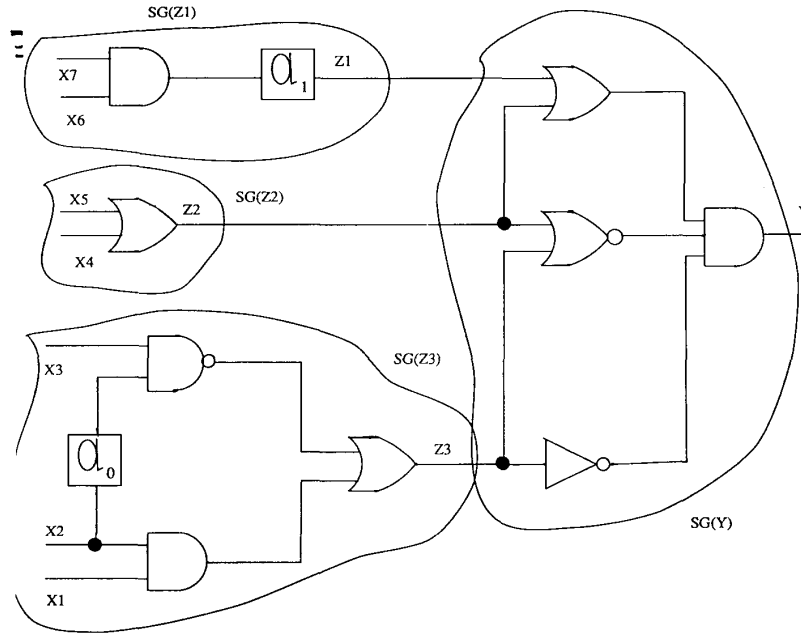


Fig. 4. Supergate structure of a circuit.

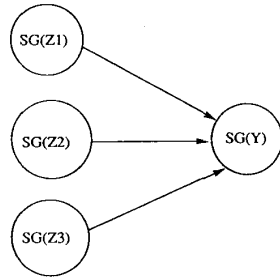


Fig. 5. Reduced circuit graph of Fig. 4.

TABLE VI

	0	1	d	$\bar{d}$
Z_1	0	$\frac{1}{4}$	0	$\frac{3}{4}$
Z_2	$\frac{1}{4}$	$\frac{3}{4}$	0	0
Z_3	0	0	0	$\frac{1}{8}$
Y	$\frac{127}{128}$	0	$\frac{1}{128}$	0

Proof: For each input X_i of X^T that fans out let $B(i, 1), \dots, B(i, N_i)$ be its fan-out branches. Then, $\text{Prob}\{X = a \parallel X_1 = a_1, \dots, X_K = a_K\}$ is equal to $\text{Prob}\{X = a\}$ computed under the following assumptions.

a) X^T is a fan-out-free circuit with inputs $Y_1, \dots, Y_P, B(1, 1), \dots, B(1, N_1), \dots, B(K, 1), \dots, B(K, N_K)$.

b) $\text{Prob}\{B(i, j) = a_i\} = 1$, for all $1 \leq j \leq N_i$ and $1 \leq i \leq K$.

c) $\text{Prob}\{B(i, j) = b\} = 0$, for all $b \in \{0, 1, d, \bar{d}\}$, $b \neq a_i$, $1 \leq j \leq N_i$ and $1 \leq i \leq K$.

Note that by assigning constants to the fan-out inputs we are factoring out all the dependencies. Using the implication rules the constants can be eliminated and we are left with a fan-out-free circuit. The result follows from the fact that the probability distribution of the output of an n input fan-out free circuit can be computed using $O(n)$ arithmetic operations [1], [10]. $\square$

Next we show how the computation of the conditional probability stated in the statement of Lemma 4.1 can be used for CDP. Let $H_1, \dots, H_N$ be the set of H-Assignments of the supergate with output X . Then, for all $a \in \{0, 1, d, \bar{d}\}$

$$\begin{aligned} \text{Prob}\{X = a\} &= \sum_{i=1}^N \text{Prob}\{(X = a) \wedge H_i\} \\ &= \sum_{i=1}^N \text{Prob}\{X = a \parallel H_i\} \times \text{Prob}\{H_i\}. \end{aligned} \quad (5)$$

Let H_i equal $\langle R_1 = a_1, \dots, R_K = a_K \rangle$. Then,

$$\begin{aligned} \text{Prob}\{H_i\} &= \text{Prob}\{R_1 = a_1, \dots, R_K = a_K\} \\ &= \text{Prob}\{R_K = a_K \parallel R_1 = a_1, \dots, R_{K-1} = a_{K-1}\} \\ &\quad \times \text{Prob}\{R_1 = a_1, \dots, R_{K-1} = a_{K-1}\} \\ &= \dots \\ &= \text{Prob}\{R_1 = a_1\} \\ &\quad \times \prod_{j=2}^K \text{Prob}\{R_j = a_j \parallel R_1 = a_1, \dots, R_{j-1} = a_{j-1}\} \\ &= \beta_1 \times \prod_{j=2}^K \beta_j. \end{aligned} \quad (6)$$

In (6), $\beta_1 = \text{Prob}\{R_1 = a_1\}$ and for all $j \geq 2$,

$$\beta_j = \text{Prob}\{R_j = a_j \parallel R_1 = a_1, \dots, R_{j-1} = a_{j-1}\}. \quad (7)$$

Let for all j , m_j be the number of gates in the MTS R_j^T , m' be the number of gates in the MTS X^T where X is the output of the supergate, and m be the number of gates in the supergates. From Lemma 4.1 it follows that for all $j \geq 2$, β_j can be computed using $O(m_j)$ arithmetic operations, and $\text{Prob}\{X = a \parallel H_i\}$ can be computed using $O(m')$ arithmetic operations. Therefore, for all i , $\text{Prob}\{(X = a) \wedge H_i\}$ can be computed using $O(m)$ arithmetic operations.

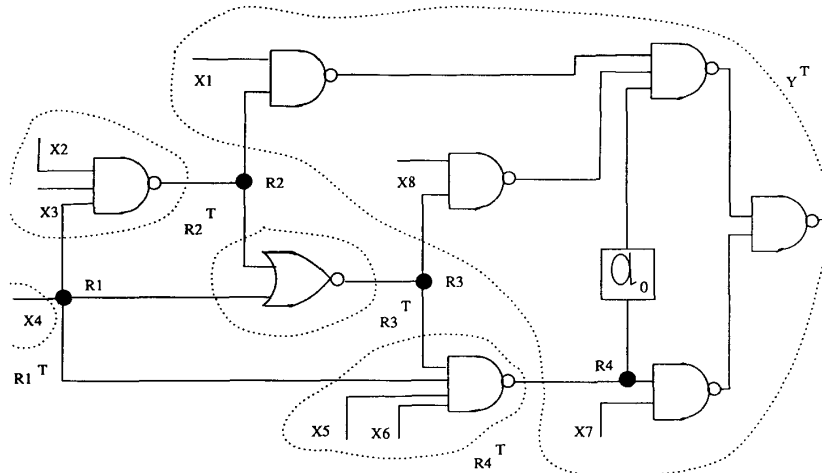


Fig. 6. Illustration of tree decomposition.

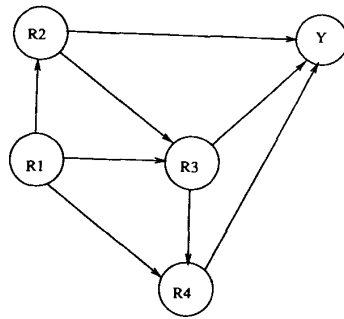


Fig. 7. Fan-out graph of the circuit of Fig. 6.

By definition, an H-Assignment H_i is inconsistent if and only if $\text{Prob}\{H_i\}$ equals 0. Therefore, from (5) it follows that we need to consider only consistent H-Assignments. Question is: how do we enumerate only the consistent H-Assignments? Algorithm H-ASS described below is an algorithm for SG-PROB that enumerates and processes only consistent H-Assignments.

H-ASS uses the notion of *consistent partial H-Assignments*. Let $\langle R_1, \dots, R_K \rangle$ be a total order of the set of fan-out points of the circuit. Then, $\langle R_1 = a_1, \dots, R_i = a_i \rangle$ is a consistent partial H-Assignment if and only if: 1) $i \leq K$, 2) for all $i \leq j \leq i$, $a_j \in \{0, 1, d, \bar{d}\}$, and 3) $\text{Prob}\{R_1 = a_1, \dots, R_i = a_i\} > 0$. Let $A = \langle R_1 = a_1, \dots, R_I = a_I \rangle$ and $B = \langle R_1 = b_1, \dots, R_J = b_J \rangle$ be two consistent partial H-Assignments. Then, B is an *extension* of A if and only if: 1) $J > I$ and 2) for all $1 \leq i \leq I$, $a_i = b_i$. If B is an extension of A then A is said to be a *subassignment* of B . The *size* of the partial H-Assignment $\langle R_1 = a_1, \dots, R_I = a_I \rangle$ is I .

Lemma 4.2: Let A be a partial H-Assignment of size I . Then, A is a consistent partial H-Assignment if and only if all subassignments of A are consistent.

Proof: Readily follows from the derivation of (6). $\square$

Lemma 4.2 forms the basis for H-ASS. H-ASS starts by enumerating a consistent partial assignment of size 1 and then finds consistent extensions of the partial assignments until it has enumerated a consistent H-Assignment. The details are presented below.

Algorithm H-ASS: (*An algorithm for SG-PROB that processes only consistent H-Assignments. In the pseudo code to

follow if the probability distribution of a point Y is stored in a one-dimensional array PY of length 4 then, $PY[1]$, $PY[2]$, $PY[3]$, $PY[4]$ contains the probability $\{Y = 0\}$, $\{Y = 1\}$, $\{Y = d\}$, and $\{Y = \bar{d}\}$, respectively. *)

Types TREE-PTR: Pointer to a tree circuit;

MTS-LIST: it consists of two fields.

a) TR: TREE-PTR;

b) NEXT: MTS-LIST-PTR.

MTS-LIST-PTR: Pointer to MTS-LIST;

CONST-LIST: it consists of two fields.

a) VAL: can have one of the four values $\{0, 1, d, \bar{d}\}$;

b) NEXT: CONST-LIST-PTR;

CONST-LIST-PTR: Pointer to CONST-LIST;

Global Variables:

PROB-DIST. An ARRAY [1..4] of real. It contains the probability distribution of Y computed so far, where Y is the output of the supergate.

ASS. A variable of type CONST-LIST-PTR. The partial assignment that is being traced out is maintained as a list and ASS points to the head of the list. If ASS contains I entries, and the VAL field of the j th entry is a_j then, $\langle R_1 = a_1, \dots, R_I = a_I \rangle$ is the partial assignment that is being traced out.

BEGIN (*MAIN*)

FP: A variable of type MTS-LIST-PTR. It is assumed that the set of MTS's of the circuit is maintained as a list. FP points to the head of the list. We assume that the total ordering of the set consisting of the fan-out points and the output of the given circuit is $\langle R_1, \dots, R_K, Y \rangle$. Accordingly, the TR field of the i th entry of FP points to the MTS R_i^T with output R_i .

For $I = 1$ TO 4 DO $\text{PROB-DIST}[I] = 0$;

ASS = NIL;

CALL H-ASS(FP, 1, ASS).

(*PROB-DIST now contains the required value of the probability distribution of the output of the supergate*).

END (*MAIN*)

PROCEDURE H-ASS (PTR: MTS-LIST-PTR; PR-ASS: real;

ASS: CONST-LIST-PTR);

EXT-ASS: CONST-LIST; P-DIST: ARRAY [1..4] of REAL;

```

BEGIN (*H-ASS*)
IF (PTR not empty) THEN
BEGIN
*1 TREE(PTR, ASS, P-DIST);
  IF (PTR.NEXT is empty) THEN
*2 FOR I = 1 TO 4 DO
    PROB-DIST[I] = P-DIST[I]*PR-ASS + PROB-
    DIST[I];
  ELSE
  BEGIN
*3 FOR J = 1 to 4 DO
    BEGIN
*4 IF (P-DIST[J] > 0) THEN
    BEGIN
      CASE J OF
        1: EXT-ASS.VAL = 0;
        2: EXT-ASS.VAL = 1;
        3: EXT-ASS.VAL = d;
        4: EXT-ASS.VAL = d;
      ENDCASE;
      APPEND EXT-ASS to end of ASS;
*5 H-ASS(PTR.NEXT, PR-ASS*P-DIST[J], ASS);
*6 REMOVE EXT-ASS from ASS;
*7 END
    END
  END
END
END (*H-ASS*)

PROCEDURE TREE (PTR: MTS-LIST-PTR; PR-DIST:
  ARRAY[1..4] of REAL; ASS: CONST-LIST-PTR);
  BEGIN (*TREE*)
    If ASS is empty then PTR is FP and PTR.TR points to  $R_1^T$ 
    which is a fan-out-free circuit. The probability distribution
    of  $R_1$  is computed by a breath first traversal of  $R_1^T$  and
    returned in PR-DIST.
    If ASS is not empty then perform the following.
    • Assume that when this procedure is invoked PTR points to
    the  $I$ th entry of the list pointed to by FP for some  $I > 0$ .
    That is, PTR.TR points to the MTS  $R_I^T$ .
    • ASS represents the partial assignment  $\langle R_1 = a_1, \dots, R_{I-1} = a_{I-1} \rangle$ .
    • Compute the probability of  $\{R_I = a\}$ , for all  $a \in \{0, 1, d, \bar{d}\}$  as discussed in the proof of Lemma 5.2.1.
    Return the probability distribution of  $R_I$  so computed in
    PR-DIST.
  END (*TREE*)
END OF ALGORITHM H-ASS.

```

A close examination of H-ASS shows that it processes only consistent assignments and each consistent assignment can be processed using $O(m)$ arithmetic operations where m is the number of gates in the circuit. We therefore have the following lemma.

Lemma 4.3: Let M be the number of consistent H-Assignments of the supergate and m be the number of gates in the supergate. Then, H-ASS computes the probability distribution of the output of the given supergate using $O(M \times m)$ arithmetic operations.

From the basic steps of our algorithm and Lemma 4.3 the following theorem follows.

Theorem 4.4: Let $SG(Z_1), \dots, SG(Z_G)$ be the supergates of the given circuit, for all $1 \leq i \leq G$, N_i be the number of gates

in $SG(Z_i)$, $N = \max_{i=1}^G N_i$, M_i be the number of consistent H-Assignments of $SG(Z_i)$, $M = \max_{i=1}^G M_i$, and m be the total number of gates in the circuit. Then, our algorithm computes the detection probability using $O(M \times m)$ arithmetic operations.

In Section I, we had made a claim regarding the algorithm for computing detection probability presented in [21]. In [21], an algorithm for the following problem was presented.

Given: a supergate $SG(X)$, for all inputs X_i of $SG(X)$ the probability of $\{X_i = 0\}$, $\{X_i = 1\}$, where $\text{Prob}\{X_i = 0\} + \text{Prob}\{X_i = 1\} = 1$, and a line l in the supergate $SG(X)$. Compute the probability of detecting at the output of $SG(X)$ the faults l stuck-at 1 and l stuck-at 0.

The above problem can be used for computing the exact value of the detection probability of faults if the fault is such that: 1) line l is stuck-at 0/1, 2) l is in the supergate $SG(Y)$, and 3) Y is an output of the circuit. In [21] it was not shown how the solution to the above problem can be used to compute the exact value of the detection probability of the stuck-at faults that do not satisfy the above constraints.

An algorithm that enumerates and processes all the input vectors can be used for computing the detection probability of faults. We refer to this as the I-ASS method. In Lemma 4.3, $M \leq 4^f$, where f is the number of fan-out points. Now f could be large enough so that 4^f is much larger than 2^n where n is the number of input variables. It appears that I-ASS can outperform our algorithm for a class of circuits. We next show that this can never happen.

Lemma 4.5: Let M be the number of consistent H-assignments and n be the number of inputs. Then, $M \leq 2^n$.

Proof: Define a binary relation $\equiv$ on the set of input vectors as follows: for all input vectors u, v $u \equiv v$ if and only if they result in the same values at the fan-out points. Clearly, $\equiv$ is an equivalence relation on the set of input vectors. Also, from the definition of consistent H-assignments we see that there exists a one-one correspondence between the equivalence classes of $\equiv$ and the set of consistent H-assignments. Therefore, $M \leq \text{no. of equivalence classes of } \equiv < 2^n$. $\square$

We thus see that I-ASS never outperforms our algorithm. On the other hand, our algorithm often does substantially better than I-ASS.

V. AN ALGORITHM FOR CSP

We show that our algorithm for CDP, with a minor modification, can also be used for CSP. The resulting algorithm is shown to be more efficient than the algorithm for CSP discussed in [21].

The basic steps of our algorithm for CSP are the same as the basic steps of our algorithm for CDP discussed in Section IV. In step 4, however, we only require the probability of $\{Y = 1\}$. The modification consists of using two-valued logic rather than four-valued logic. The definitions of F-Assignments and H-Assignments are changed accordingly. The probability distribution of point Z now consists of the probability of the events $\{Z = 0\}$, $\{Z = 1\}$, and the definition of SG-PROB is changed accordingly. All arrays of size 4 in H-ASS are changed to arrays of size 2, and the CASE STATEMENT in procedure H-ASS now consists of only the first two cases.

In [21] an algorithm for computing the exact value of signal probability was presented. The algorithm in [21] and our algorithm have the same basic steps. The two algorithms differ in the solutions to SG-PROB. Our algorithm uses H-ASS for SG-PROB

whereas, the algorithm in [21] uses F-ASS for SG-PROB. F-ASS is discussed below.

F-ASS solves SG-PROB by enumerating and processing consistent *F-Assignments* as discussed below. The set of inputs of any supagate $SG(X)$ can be subdivided into two sets.

1) $INF(X)$, the set of *nonfan-out inputs* of $SG(X)$, consisting of those inputs of $SG(X)$ which have exactly one path to X .

2) $IF(X)$, the set of *fan-out inputs* of $SG(X)$, consisting of those inputs of $SG(X)$ which have more than one path to X .

In Fig. 4, $INF(Y) = \{Z_1\}$ and $IF(Y) = \{Z_2, Z_3\}$. In order for an input to be a fan-out input it need not be a fan-out point. The circuit in Fig. 6 consists of only one supagate $SG(Y)$ for which $IF(Y) = \{X_2, X_3, X_4, X_5, X_6\}$ and $INF(Y) = \{X_1, X_7, X_8\}$. Note that X_2 is a fan-out input of $SG(Y)$ but not a fan-out point.

An *F-Assignment* of a supagate $SG(X)$ is an ordered tuple $\langle X_1 = a_1, \dots, X_K = a_K \rangle$ where 1) for all i , $a_i \in \{0, 1\}$ and 2) $\{X_1, \dots, X_K\}$ equals $IF(X)$. For a probability distribution of the inputs of a supagate, an *F-Assignment* $\langle X_1 = a_1, \dots, X_K = a_K \rangle$ is said to be a *consistent F-Assignment* if and only if for all $1 \leq i \leq K$, $\text{Prob}\{X_i = a_i\} \neq 0$. Let $A_1, \dots, A_M$ be the consistent *F-Assignments* of $SG(X)$. Then, for all $a \in \{0, 1\}$ we have

$$\text{Prob}\{X = a\} = \sum_{i=1}^M [\text{Prob}\{X = a \mid A_i\} \times \text{Prob}\{A_i\}]. \quad (8)$$

Let A_i equal $\langle X_1 = a_1, \dots, X_K = a_K \rangle$. Then,

$$\text{Prob}\{A_i\} = \prod_{j=1}^K \text{Prob}\{X_j = a_{ij}\}. \quad (9)$$

Since for all j the values of $\text{Prob}\{X_j = a_j\}$ are known, $\text{Prob}\{A_i\}$ can be computed using (9) above. Also, by assigning constants to the fan-out inputs, $SG(X)$ is converted into a fan-out-free circuit. Therefore, $\text{Prob}\{X = a \mid A_i\}$ can be computed using a linear number of arithmetic operations. Therefore, each term of (8) can be computed using a linear number of arithmetic operations. We have the following lemma.

Lemma 5.1: The number of arithmetic operations required by F-ASS to solve an instance of SG-PROB is $O(M \times m)$ where M is the number of consistent *F-Assignments* and m is the number of gates in the supagate.

We next show that H-ASS is more efficient than F-ASS. Observe the following.

1) Assigning constants to the fan-out inputs of a supagate fixes the values of all the fan-out points of the supagate. The values at the nonfan-out inputs of a supagate do not affect the value at the fan-out points of the supagate.

2) An H-Assignment H_i is consistent if and only if there exists a consistent *F-Assignment* F_j such that the value at the fan-output point resulting due to F_j corresponds to the H-Assignment H_i .

Lemma 5.2: Let M_1 be the number of consistent *F-Assignments* of a supagate and M_2 be the number of consistent H-Assignments of a supagate. Then, $M_1 \geq M_2$.

Proof: Let CFA be the set of consistent *F-Assignments* of the supagate and CHA be the set of consistent H-Assignments of the supagate. Define a *binary relation* $\equiv$ on CFA as follows: for all $X, Y \in \text{CFA}$, $X \equiv Y$ if and only if the consistent H-Assignment resulting due to X, Y are the same. It can be verified that $\equiv$ is an equivalence relation. Therefore, $\|\text{CFA}\| \geq \|\text{CHA}\|$. $\square$

From Lemmas 5.1 and 5.2 it follows that H-ASS is more efficient than F-ASS. In fact, H-ASS is often considerably more efficient than F-ASS. An example will substantiate this claim. In Fig. 6 assume that the circuit is fault-free, i.e., the pseudo gates do not exist and we want to compute the signal probability of the output. The number of consistent *F-Assignments* of the supagate of Fig. 6, required by F-ASS, is 32 because the supagate has five fan-out inputs. Compared to this, the number of consistent H-Assignments is only 3. We thus see that our algorithm for CSP is never worse than the algorithm for CSP presented in [21] and is often substantially better.

VI. CONCLUSION

CDP and CSP are two important problems in the domain of VLSI testing. We presented an algorithm for CDP which uses the notion of supagates [21], [22], four-valued logic [19], consistent H-assignments, and pseudo gates. The use of pseudo gates makes it possible for the algorithm to be used for multiple stuck-at faults in circuits with multiple outputs. One can compute the exact value of detection probability by enumerating and processing each input vector. We show that our algorithm never performs worse than such an algorithm and often performs substantially better.

The algorithm for CDP presented here uses an efficient enumeration algorithm that enumerates only the consistent H-assignments. We showed how this enumeration algorithm can also be used to improve the algorithm for CSP presented in [21] and [22].

By using a different pseudo gate the algorithm for CDP can be used for computing the detection probability of nonfeedback bridging faults [26]. A similar algorithm can be used for a number of other problems [27].

APPENDIX

PROBABILITY EXPRESSIONS OF THE PRIMITIVE GATES

$X(a)$ denotes the probability of the event $\{X = a\}$.

α_0 : F is the input and G is the output of the gate.

$$G(0) = F(0) + F(\bar{d})$$

$$G(d) = F(d) + F(1)$$

$$G(1) = G(\bar{d}) = 0.$$

α_1 : IF is the input and G is the output of the gate.

$$G(0) = G(d) = 0$$

$$G(1) = F(d) + F(1)$$

$$G(\bar{d}) = F(0) + F(\bar{d}).$$

Θ : Z_1 and Z_2 are the inputs and Z is the output.

$$Z(0) = Z(\bar{d}) = 0$$

$$Z(1) = \{Z_1(0) + Z_1(1)\} \times \{Z_2(0) + Z_2(1)\}$$

$$Z(d) = 1 - Z(1).$$

NOT: Z_1 is the input and Z is the output.

$$Z(0) = Z_1(1)$$

$$Z(d) = Z_1(\bar{d})$$

$$Z(\bar{d}) = Z_1(d)$$

$$Z(1) = Z(0).$$

AND: Z_1 and Z_2 are the inputs and Z is the output of the gate.

$$Z(1) = Z_1(1) \times Z_2(1)$$

$$Z(\bar{d}) = \{Z_1(\bar{d}) + Z_1(1)\} \times \{Z_2(\bar{d}) + Z_2(1)\} - Z(1)$$

$$Z(d) = \{Z_1(d) + Z_1(1)\} \times \{Z_2(d) + Z_2(1)\} - Z(1)$$

$$Z(0) = 1 - Z(1) - Z(d) - Z(\bar{d}).$$

OR: Z_1 and Z_2 are the inputs and Z is the output of the gate.

$$Z(0) = Z_1(0) \times Z_2(0)$$

$$Z(\bar{d}) = \{Z_1(\bar{d}) + Z_1(0)\} \times \{Z_2(\bar{d}) + Z_2(0)\} - Z(0)$$

$$Z(d) = \{Z_1(d) + Z_1(0)\} \times \{Z_2(d) + Z_2(0)\} - Z(0)$$

$$Z(1) = 1 - Z(0) - Z(d) - Z(\bar{d})$$

ACKNOWLEDGMENT

We thank the reviewers for their helpful comments. We also acknowledge the discussions we had with Prof. D. Arden and R. Murray during the early stages of this research.

REFERENCES

- [1] P. Agrawal and V. D. Agrawal, "Probabilistic analysis of random test generation methods for irredundant combinational logic networks," *IEEE Trans. Comput.*, vol. C-24, no. 7, pp. 691-695, July 1975.
- [2] V. D. Agrawal and S. C. Seth, "Probabilistic testability," in *Proc. 1985 Int. Conf. Comput. Design: VLSI Computers*, pp. 562-565.
- [3] V. D. Agrawal, S. C. Seth, and C. C. Chuang, "Probabilistically guided test generation," in *Proc. 1985 Int. Symp. Circuits Syst.*, pp. 687-690.
- [4] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*. Reading, MA: Addison-Wesley, 1974.
- [5] B. B. Bhattacharya and S. C. Seth, "On the reconvergent structure of combinational circuits and application to compact testing," in *Proc. 17th Int. Fault-Tolerant Comput. Symp.*, 1987, pp. 264-269.
- [6] M. A. Breuer and A. D. Friedman, *Diagnosis and Reliable Design of Digital Systems*. Rockville, MD: Computer Science Press, 1976.
- [7] S. Chakravarty, "On the testing, reliability analysis and synthesis of combinational circuits," Ph.D. dissertation, Dep. Comput. Sci., State University of New York at Albany, Dec. 1986.
- [8] —, "A note on random versus deterministic testing," in *Proc. 1987 Int. Conf. Comput. Aided Design*, pp. 152-155.
- [9] —, "Computing detectability is harder than computing test vectors," Tech. Rep. 87-17, Dep. Comput. Sci., State University of New York at Buffalo, Buffalo, NY 14260.
- [10] S. Chakravarty and H. B. Hunt III, "On the computation of detection probability for multiple faults," in *Proc. 1986 Int. Test Conf.*, pp. 252-262.
- [11] C. Chin and E. J. McCluskey, "Testlength for pseudorandom testing," *IEEE Trans. Comput.*, vol. C-36, pp. 252-256, Feb. 1987.
- [12] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: Freeman, 1979.
- [13] R. K. Gaede, M. R. Mercer, and B. Underwood, "Calculation of greatest lower bounds obtainable by the Cutting Algorithm," in *Proc. 1986 Int. Test Conf.*, pp. 498-505.
- [14] O. H. Ibarra and S. K. Sahni, "Polynomially complete fault detection problem," *IEEE Trans. Comput.*, vol. C-24, no. 3, pp. 242-249, 1975.
- [15] S. K. Jain and V. D. Agrawal, "STAFAN: An alternative to fault simulation," in *Proc. 21st Design Automat. Conf.*, 1984, pp. 18-23.
- [16] B. Krishnamurthy and I. G. Tollis, "Improved techniques for estimating signal probabilities," in *Proc. 1986 Int. Test Conf.*, pp. 244-251.
- [17] K. P. Parker and E. J. McCluskey, "Analysis of faults using input signal probabilities," *IEEE Trans. Comput.*, vol. C-24, no. 5, pp. 573-578, May 1975.
- [18] K. P. Parker and E. J. McCluskey, "Probabilistic treatment of general combinational networks," *IEEE Trans. Comput.*, vol. C-24, no. 6, pp. 668-670, June 1975.
- [19] J. P. Roth, W. G. Bouricius, and P. R. Schneider, "Programmed algorithms to compute tests to detect and distinguish between failures in logic circuits," *IEEE Trans. Electron. Comput.*, vol. EC-16, no. 10, pp. 567-580.
- [20] J. Savir, G. S. Ditlow, and P. H. Bardell, "Random pattern testability," *IEEE Trans. Comput.*, vol. C-33, no. 1, pp. 79-90, Jan. 1984.
- [21] S. C. Seth, B. B. Bhattacharya, and V. D. Agrawal, "An exact analysis for efficient computation of random pattern testability in combinational circuits," in *Proc. 16th Int. Fault-Tolerant Comput. Symp.*, 1986, pp. 318-323.
- [22] S. C. Seth, L. Pan, and V. D. Agrawal, "PREDICT: Probabilistic estimation of digital circuit testability," in *Proc. 15th Int. Fault-Tolerant Comput. Symp.*, pp. 220-225.
- [23] K. D. Wagner, C. K. Chin, and E. J. McCluskey, "Pseudorandom testing," *IEEE Trans. Comput.*, vol. C-36, no. 3, p. 332-343, Mar. 1987.
- [24] T. W. Williams and K. P. Parker, "Design for testability—A survey," *IEEE Trans. Comput.*, vol. C-31, no. 1, pp. 2-15, Jan. 1982.
- [25] W. H. Wunderlich, "PROTEST: A tool for probabilistic testability analysis," in *Proc. 22nd Design Automat. Conf.*, 1985, pp. 204-211.
- [26] S. Chakravarty, "Testing of non-feedback bridging faults," *Integration: The VLSI J.*, no. 9, pp. 109-127, 1990.
- [27] S. Chakravarty and H. B. Hunt III, "On the generalized probability problem with application to testing and reliability analysis," Tech. Rep. Buffalo, 87-06, Dep. Comput. Sci., State University of New York, Buffalo, NY 14260.



Sreejit Chakravarty was born in Shillong, India, in 1958. He graduated from the Birla Institute of Technology and Science, Pilani, India, in 1980 with the B.Tech. degree in electrical and electronics engineering. During 1980-1981 he was a Development Engineer at Tata Electric Company's R&D Labs, Bombay, India. He subsequently graduated with the M.S. degree in computer science from The State University of New York, Stonybrook in 1983 and the Ph.D. degree in computer science from The State University of New York, Albany, in 1986.

Since then he has been an Assistant Professor of Computer Science at The State University of New York, Buffalo. His areas of interest are CAD aspects of VLSI, analysis of algorithms, and fault-tolerant architectures.

Harry B. Hunt III was born in Sharon, PA, in 1949. He received the B.S. degree in mathematics with highest honors and the M.S. degree in mathematics from Case Western Reserve University, Cleveland, OH, in 1971, and the Ph.D. degree in computer science from Cornell University, Ithaca, NY, in 1973.

He is currently a Professor of Computer Science at the State University of New York at Albany. His primary research interests are in computational complexity, analysis of algorithms, discrete mathematics, combinatorial optimization, and the design and testing of digital circuits.